



Secure Kubernetes Production Architecture

Self-hosted HA platform on Proxmox
Talos Linux + Kubernetes + Cilium + Istio Ambient

Implementation Architecture v4.1

30 August 2026 | Idaho, United States | Confidential

Decision summary: retain Talos Linux, upstream Kubernetes, Cilium, Istio Ambient, Flux and Flagger;
move CockroachDB, Qdrant, Redis and NATS into the cluster data plane; adopt Longhorn and Velero;
expand observability with Tempo, Pyroscope, Faro and optional Beyla; add Headlamp plus core Kubernetes
add-ons: cloud-controller-manager, Metrics Server, NodeLocal DNSCache, Descheduler and CSI Snapshot Controller.



Document status

Supersedes v4.0

This document supersedes Aerealith Secure Kubernetes Production Architecture v4.0. The v4.1 update adds five cluster add-ons to the production baseline: an external cloud-controller-manager for Proxmox integration, Metrics Server for the Kubernetes resource Metrics API and autoscaling, NodeLocal DNSCache for node-local DNS resilience/performance, Descheduler for policy-driven rebalancing, and the CSI Snapshot Controller for cluster-wide VolumeSnapshot lifecycle. The v4.0 data, Longhorn, Velero, observability, Headlamp, Flux monorepo, and environment/preview decisions remain in force.

This remains an implementation architecture and security baseline. It supports audit readiness for ISO/IEC 27001, SOC 2, HIPAA Security Rule safeguards, and GDPR accountability, but it does not create certification, attestation, or legal compliance by itself.

Field	Value
Architecture owner	TBD
Security approver	TBD
Service owner	Aerealith product owner - TBD
Version	v4.1
Review cadence	Quarterly and after material architecture, threat, data-classification, dependency, or regulatory change
Change control	Protected GitHub pull request with retained CI validation, security evidence, and approval

v4.1 change summary

- Add an external cloud-controller-manager for Proxmox integration, with leader election, least-privilege Proxmox API credentials, providerID/node lifecycle handling, and topology labels. Treat it as a control-plane add-on even when its pods are scheduled on platform workers.
- Add Metrics Server as the resource Metrics API provider for kubectl top and CPU/memory-based HPA/VPA inputs. Prometheus remains the monitoring and historical metrics system.
- Add NodeLocal DNSCache as a kube-system DaemonSet so workloads query a node-local cache before CoreDNS, reducing cross-node DNS latency and contrack pressure.
- Add Kubernetes Descheduler as a policy-controlled CronJob/Deployment in kube-system to correct long-lived placement drift while respecting PDBs, priorities, topology, and explicit exclusions for quorum/stateful workloads.
- Add the CSI Snapshot Controller and VolumeSnapshot CRDs as cluster-wide storage primitives. Longhorn remains the CSI implementation and Velero/native backups remain the recovery authority; snapshots alone are not backups.
- CockroachDB is now self-hosted inside Kubernetes using the current CockroachDB Operator; the cloud-hosted database is no longer the target production architecture.
- Qdrant, Redis, and NATS run inside the cluster alongside CockroachDB under the data security zone.
- Longhorn becomes the Kubernetes CSI storage layer, with workload-specific StorageClasses so replicated databases do not blindly double-replicate every block. The cluster-wide CSI Snapshot Controller and VolumeSnapshot CRDs provide the standard snapshot API consumed by Longhorn and Velero workflows.
- Velero becomes a required Kubernetes resource/PVC backup layer and complements, rather than replaces, native database snapshots and physical Proxmox Backup Server recovery.
- Observability replaces Jaeger with Tempo and adds Pyroscope, Grafana Faro, a narrowly scoped Grafana Alloy Faro receiver, and optional Grafana Beyla.
- Headlamp is added as the private Kubernetes operations UI.
- Flux remains the production reconciler, but the GitOps tree now lives in the Aerealith monorepo under infra/talos and infra/kubernetes, organized by namespace and application.
- Single-cluster environment isolation is formalized with aerealith-dev, aerealith-staging, aerealith-prod, and ephemeral preview-pr-* namespaces.

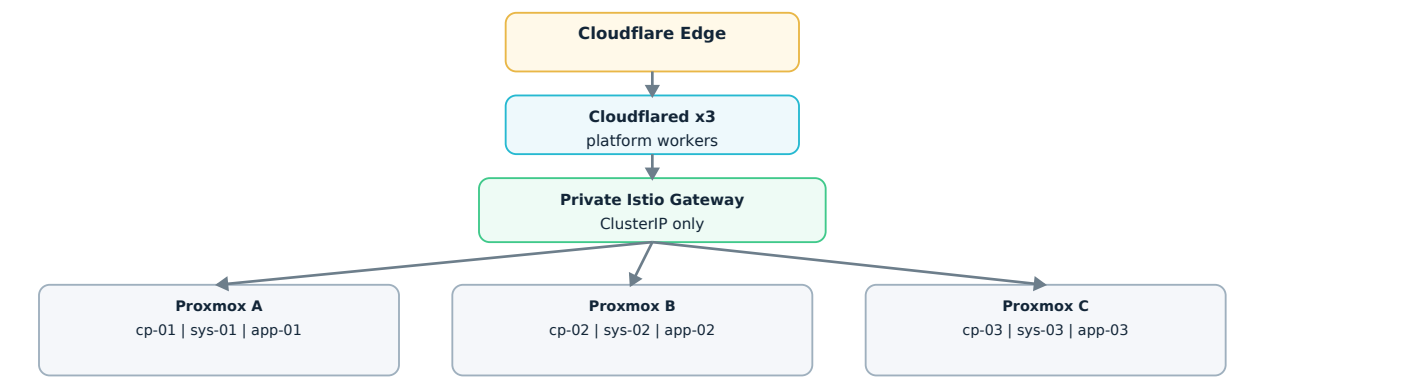
Table of contents

Document status	2
v4.1 change summary	2
Table of contents	3
1. Executive decision	5
2. Kubernetes distribution selection	6
3. Threat model and trust boundaries	7
Defense-in-depth layers	7
4. Physical placement and node pools	8
5. Address plan and network services	9
6. Talos Linux host hardening	10
7. Control-plane hardening	11
8. API, etcd, and node firewall policy	12
9. Cilium network security plane	13
10. Istio Ambient service security	14
11. Platform applications and namespace map	15
Core Kubernetes add-on operating rules	15
12. Environment and preview namespace model	16
Preview controls	16
13. Default-deny network policy model	17
14. Identity, RBAC, and administrative access	18
15. Pod Security and Kyverno admission policy	19
16. Software supply-chain security	20
17. Self-hosted OCI registry strategy	21
18. Secrets management and PKI	22
19. Storage and persistence strategy	23
20. Data platform: CockroachDB, Qdrant, Redis, and NATS	24
Environment isolation inside the shared data plane	24
21. Backup and disaster recovery	25
22. Canary and progressive delivery	26
23. Flux GitOps monorepo operating model	27
Reconciliation order	27
24. GitHub CI/CD integration	28
Repository protections	28
25. Cloudflare Tunnel and private ingress	29
26. Runtime security and posture management	30
27. Observability and telemetry architecture	31
28. Exporters and scrape matrix	32
29. Audit logging and evidence-quality telemetry	33

30. CockroachDB security and Kubernetes placement	34
31. Failure-mode analysis	35
32. Incident containment and recovery	36
33. Patch, upgrade, and vulnerability management	37
34. Vulnerability-management SLA	38
35. Security standards and compliance mapping	39
36. Automated compliance evidence	40
37. Phased implementation plan	41
38. Production validation checklist	42
Cluster foundation	42
Core Kubernetes add-ons	42
Network, mesh, and environments	42
Workload and supply-chain security	42
Data, storage, and recovery	42
Observability and operations	42
39. Definitive architecture	43
A. Configuration baselines	44
Control-plane VIP document	44
Platform worker labels and taints	44
Application worker labels	44
Critical workload spread	44
Environment namespace baseline	44
B. Port and firewall matrix	45
C. Component maturity and selection	46
D. Authoritative references	47

1. Executive decision

Aerealith should run on a deliberately hardened, self-hosted Kubernetes platform where the node operating system, cluster control plane, software supply chain, data services, and observability pipeline are all part of the security boundary. The selected baseline remains Talos Linux + upstream Kubernetes. Control-plane nodes are dedicated and do not run normal workloads.



The cluster uses three dedicated control-plane VMs, three platform workers, and three application workers, spread one of each across three physical Proxmox hosts. Stateful data services run in Kubernetes but are topology-spread across physical failure domains. If database I/O or capacity later justifies it, a three-node data worker pool can be added without changing the logical design.

Layer	Selected design	Security / reliability purpose
Node OS	Talos Linux	Immutable/minimal host; no SSH or package manager; API-only administration
Kubernetes	Upstream Kubernetes on Talos	Standard APIs and conformance without lightweight-distro constraints
Core add-ons	Proxmox CCM + Metrics Server + NodeLocal DNSCache + Descheduler + CSI Snapshot Controller	Provider integration, autoscaling metrics, DNS cache, placement correction, and snapshot API lifecycle
Control plane	3 dedicated VMs + embedded etcd	2/3 quorum after one physical-host loss; no normal workloads
Platform workers	3 VMs	Ingress, GitOps, security, secrets, observability, storage control plane
Application workers	3 VMs	Aerealith and data workloads; resource and blast-radius separation
CNI	Cilium + Hubble + WireGuard	L3/L4 identity policy, flow visibility, node/pod encryption
Service mesh	Istio Ambient	Workload identity, mTLS, L7 authorization and traffic shifting
GitOps	Flux CD	Pull-based reconciliation from the monorepo
Progressive delivery	Flagger	Prometheus-gated canary promotion and rollback
Secrets / PKI	SOPS + OpenBao + ESO + cert-manager	Separate bootstrap authority, runtime secrets, and certificate lifecycle
Storage	Longhorn + workload-specific StorageClasses	Kubernetes-native CSI with deliberate replication policy
Data	CockroachDB + Qdrant + Redis + NATS	Self-hosted primary data, vector, cache, and messaging services
Backups	Velero + native snapshots + physical PBS + offsite	Independent recovery layers
Observability	OTel + Prometheus + Loki + Tempo + Grafana + Pyroscope + Faro	Metrics, logs, traces, profiles, RUM, alerts
Operations UI	Headlamp	Private RBAC-aware Kubernetes interface

2. Kubernetes distribution selection

The distribution decision optimizes host attack surface, configuration drift, CIS posture, controlled upgrades, recovery, and strong defaults rather than installation convenience. Talos remains the selected production baseline.

Option	Security posture	Operational model	Strengths	Main drawback	Decision
Talos Linux + Kubernetes	Excellent by design	API-driven immutable nodes	No SSH/shell/package manager; atomic upgrades; built-in VIP	Different operating model; fewer traditional host agents	SELECTED
RKE2	Very strong	Traditional Linux + hardened distro	CIS profile; familiar tooling	Host OS remains a large mutable surface	Fallback
kubeadm	Operator dependent	Upstream primitives	Maximum transparency/flexibility	Highest hardening and lifecycle burden	Not selected
K3s	Good when hardened	Lightweight integrated distro	Small footprint and easy HA	Resource savings are not the primary design goal	Not production baseline

Why Talos still wins

Talos removes SSH, an interactive shell, package management, and mutable root filesystem assumptions from the normal operating model. Management is authenticated through the Talos API; upgrades are image-based; the platform supports Kubernetes audit logging, encrypted secrets, mTLS to etcd, and hardened defaults.

RKE2 remains the recovery option if future requirements demand conventional Linux host agents, custom kernel tooling, or enterprise support that is operationally easier on Ubuntu/RHEL. Kubeadm remains useful as a reference implementation, but it intentionally leaves host lifecycle and most hardening choices to the operator.

3. Threat model and trust boundaries

The physical LAN remains 10.42.0.0/24 and intentionally does not use VLAN segmentation. That choice makes destination-side enforcement mandatory: a compromised workstation, appliance, BMC, VM, or container may share the same L2 broadcast domain, so each trust boundary must reject traffic it does not require. Kubernetes NetworkPolicy alone cannot protect Proxmox, network appliances, BMCs, or the Talos API.

Threat / failure	Primary controls	Residual risk
Compromised LAN client	Talos ingress firewall; Proxmox guest firewall; management VPN; TLS/mTLS	Same L2 remains discoverable; gateway cannot filter same-subnet traffic
Compromised workload	Cilium default deny; Istio mTLS/authz; PSA; Kyverno; runtime detection	Kernel/container-runtime zero-days; stolen app identity
Malicious/compromised developer token	Protected branches; CODEOWNERS; CI least privilege; signed artifacts; Flux pull reconciliation	GitHub remains a Tier-0 trust plane
Compromised image/dependency	Trivy; SBOM; Cosign; digest pinning; Kyverno verification	Unknown supply-chain attack not detected by scanners
One Proxmox host lost	etcd/data/OpenBao quorum; topology spread; PDBs; N+1 capacity	Second correlated host loss
Control-plane credential theft	Talos mTLS; management overlay; short-lived kubeconfig; offline break-glass	Tier-0 credential exposure remains severe
Ransomware/operator deletion	Velero/native snapshots; PBS; offsite/offline copy; delete-separated credentials	Site compromise before backup isolation
Preview environment escape	Namespace isolation; separate identities; quotas; no production data/secrets	Shared kernel and cluster control plane
Cloudflare outage/account issue	Private management plane remains; provider monitoring; documented edge dependency	Public app ingress unavailable without alternate edge

Defense-in-depth layers

Layer	Control intent
Physical	Locked rack, UPS, media custody, recovery equipment
Node / VM	Talos immutable OS, ingress firewall, Secure Boot/TPM where validated
Cluster	RBAC, PSA, Kyverno, encrypted secrets, restricted admin paths
Network	Cilium default deny, Hubble, WireGuard, exact egress
Service identity	Istio mTLS and AuthorizationPolicy
Supply chain	Trivy, SBOM, Cosign, digest admission
Runtime	Falco, Kubescape, optional Tetragon
Data	OpenBao, per-service TLS, application auth, retention, native backups
Recovery	Velero, etcd/OpenBao/data snapshots, PBS, offsite copy
Evidence	Audit logs, policy reports, release records, restore/failure-test evidence

4. Physical placement and node pools

The preferred production topology continues to use nine Kubernetes VMs: three control-plane, three platform workers, and three application workers. Data services now run inside Kubernetes, so application-worker CPU, memory, disk endurance, and IOPS planning must include database capacity rather than only product pods.

Node	Role	PVE	Planning vCPU	Planning RAM	Planning disk	Scheduling
k8s-cp-01	Talos control plane + etcd	PVE A	4	8 GiB	120 GiB SSD/NVMe	No normal workloads
k8s-cp-02	Talos control plane + etcd	PVE B	4	8 GiB	120 GiB SSD/NVMe	No normal workloads
k8s-cp-03	Talos control plane + etcd	PVE C	4	8 GiB	120 GiB SSD/NVMe	No normal workloads
k8s-sys-01	Platform worker	PVE A	8	24 GiB	100 GiB OS + 500 GiB+ data	Platform/security only
k8s-sys-02	Platform worker	PVE B	8	24 GiB	100 GiB OS + 500 GiB+ data	Platform/security only
k8s-sys-03	Platform worker	PVE C	8	24 GiB	100 GiB OS + 500 GiB+ data	Platform/security only
k8s-app-01	Application + data worker	PVE A	12+	48+ GiB	100 GiB OS + 1 TiB+ NVMe	Aerealith + one data replica
k8s-app-02	Application + data worker	PVE B	12+	48+ GiB	100 GiB OS + 1 TiB+ NVMe	Aerealith + one data replica
k8s-app-03	Application + data worker	PVE C	12+	48+ GiB	100 GiB OS + 1 TiB+ NVMe	Aerealith + one data replica

Capacity gate

Production is accepted only if two surviving application workers and two surviving platform workers can carry critical requests after a complete Proxmox-host loss. Because CockroachDB, Qdrant, Redis, and NATS now share the application worker pool, test one-host loss under realistic database and queue load, not only synthetic HTTP traffic.

If I/O contention or memory pressure becomes material, add three tainted data workers, one per physical host, and move the data namespace there. Do not solve capacity pressure by scheduling normal workloads on the control plane.

5. Address plan and network services

The physical LAN stays 10.42.0.0/24 and remains intentionally unsegmented by VLAN. Kubernetes must use ranges that do not overlap the LAN or remote VPN/site routes. The selected pod and service CIDRs remain unchanged.

Use	Range / value	Status
Physical LAN	10.42.0.0/24	Existing
Pod CIDR	10.200.0.0/16	Selected
Service CIDR	10.201.0.0/16	Selected
Cluster DNS	10.201.0.10	Selected
Kubernetes API VIP	10.42.0.TBD	Reserve outside DHCP
Talos API	TCP 50000 on each node	Management overlay only
Management overlay	10.250.0.0/24 proposed	Validate all remote routes
Data services	ClusterIP only	No direct LAN listener
Headlamp/Grafana/OpenBao UI	Private management path	Never public application hostnames

Important Talos recovery rule
Do not use the Kubernetes VIP as the Talos API endpoint for recovery. Keep direct per-node Talos API addresses reachable from the management overlay even if etcd or the Kubernetes API is unhealthy.

6. Talos Linux host hardening

Talos remains the largest host-security improvement over the former K3s baseline. The production objective is to make node state reproducible and reviewable, not to maintain snowflake Linux servers by SSH.

Host control	Production requirement
Remote administration	Talos API only; mTLS client identities; reachable only from management overlay and named recovery station
Root filesystem	Immutable; no package manager; no ad-hoc node changes
Ingress firewall	Default block with explicit port/source rules only
Disk encryption	Encrypt STATE and EPHEMERAL; prefer TPM-backed LUKS2 after Proxmox validation
Secure Boot	Enable UEFI Secure Boot and vTPM after restore/rollback proof
Logging	Forward Talos service and kernel logs to an independent collector path
Node config	Generated from reviewed patches; secret material outside ordinary Git
Upgrades	One node at a time; preflight; A/B rollback; supported minor path only
Debug access	Time-bounded privileged debug container only under incident/change record
Production Proxmox VM firmware Use OVMF/UEFI, q35 machine type, CPU type host, fixed RAM for control-plane/data-sensitive VMs, VirtIO storage/networking, and vTPM2 where supported. Validate Secure Boot + vTPM + PBS restore before enforcing it fleet-wide.	

7. Control-plane hardening

Control-plane availability uses the Talos layer-2 VIP and three etcd members, one per physical host. Only Kubernetes control-plane components, etcd, Cilium node networking, and explicitly approved node-critical security/telemetry DaemonSets may execute on control-plane nodes. No Aerealith app, database, Istio waypoint/gateway, OpenBao, Prometheus, Flux controller, Longhorn replica, or general platform Deployment is scheduled there.

```
cluster:
  allowSchedulingOnControlPlanes: false
  network:
    podSubnets:
      - 10.200.0.0/16
    serviceSubnets:
      - 10.201.0.0/16
  apiServer:
    admissionControl:
      - name: PodSecurity
        configuration:
          apiVersion: pod-security.admission.config.k8s.io/v1
          kind: PodSecurityConfiguration
          defaults:
            enforce: restricted
            audit: restricted
            warn: restricted
```

Control-plane rule

The control plane is a recovery and trust boundary, not spare compute. If normal product or data pods are found there, treat it as a configuration defect and fix scheduling/taints rather than documenting it as normal.

8. API, etcd, and node firewall policy

Service	Exposure	Required control
Talos API 50000/TCP	Per-node IP only	Management VPN/recovery subnet; mTLS; never public
Kubernetes API 6443/TCP	Talos VIP + node IPs	Workers, management VPN, approved collectors; OIDC/RBAC for humans
etcd 2379-2380/TCP	Control-plane node IPs only	Only control-plane /32 set; mTLS; never workers or admin LAN
Kubelet 10250/TCP	Node IPs	Control plane and approved monitoring only
Cilium VXLAN 8472/UDP	Kubernetes nodes	Kubernetes node IP set only
Cilium WireGuard 51871/UDP	Kubernetes nodes	Kubernetes node IP set only
Istio HBONE 15008/TCP	Worker nodes	Worker node set only; validate pinned release
Longhorn engine/manager ports	Kubernetes workers	Worker-node set only; never LAN-wide
Data services	ClusterIP only	Cilium identity policy + service TLS; no NodePort/LB
Headlamp/Grafana/OpenBao UI	ClusterIP/private route	VPN/OIDC only; no public app route

```

apiVersion: v1alpha1
kind: NetworkDefaultActionConfig
ingress: block
---
apiVersion: v1alpha1
kind: NetworkRuleConfig
name: talos-api-admin
portSelector:
  ports: [50000]
  protocol: tcp
ingress:
  - subnet: 10.250.0.0/24

```

Do not enable two host firewalls blindly

Talos ingress firewall owns node-service filtering. Keep Cilium Host Firewall disabled initially so nftables/BPF ordering cannot silently conflict with OS rules. Cilium NetworkPolicy still owns pod traffic.

9. Cilium network security plane

Cilium runs on every Kubernetes node, including control planes. Keep kube-proxy for the first secure release to minimize Cilium/Istio interaction risk. Use Cilium for pod reachability, L3/L4 identity policy, FQDN egress, Hubble telemetry, and WireGuard node/pod encryption.

```
kubeProxyReplacement: false
cni:
  exclusive: false
bpf:
  masquerade: false
routingMode: tunnel
tunnelProtocol: vxlan
encryption:
  enabled: true
  type: wireguard
  nodeEncryption: true
operator:
  replicas: 2
hubble:
  relay:
    enabled: true
hostFirewall:
  enabled: false
```

With Cilium under Istio Ambient, keep the platform prerequisites aligned with the pinned versions and explicitly allow the documented Ambient kubelet health-probe source where default-deny would otherwise block it.

```
apiVersion: cilium.io/v2
kind: CiliumClusterwideNetworkPolicy
metadata:
  name: allow-ambient-hostprobes
spec:
  enableDefaultDeny:
    ingress: false
    egress: false
  endpointSelector: {}
  ingress:
    - fromCIDR:
        - 169.254.7.127/32
```

10. Istio Ambient service security

Clear ownership prevents Cilium and Istio from enforcing contradictory L7 behavior. Istio Ambient remains preferred over mandatory sidecars. Ztunnel provides secure L4 transport and workload identity; waypoints are introduced only where an application needs L7 authorization or traffic policy. Tier-0 infrastructure is not meshed by default.

Responsibility	Cilium	Istio
Pod connectivity	Owner	No
L3/L4 default deny	Owner	Supplemental identity checks
Node-to-node encryption	WireGuard	No
Workload identity	Kubernetes/Cilium identity context	Owner
Service-to-service mTLS	No	Owner
HTTP method/path authorization	Avoid in meshed apps	Owner via waypoint/gateway
Ingress traffic shifting	No	Owner
FQDN egress	Owner by default	Special L7 egress cases only
Flow visibility	Hubble	Request/service telemetry

```
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: strict-mtls
  namespace: aerealith-prod
spec:
  mtls:
    mode: STRICT
```

Mesh enrollment rule
Mesh Aerealith application namespaces and explicitly selected shared services. Do not initially mesh kube-system, flux-system, openbao, cert-manager, external-secrets, longhorn-system, velero, or the control plane. Circular dependencies make recovery harder.

11. Platform applications and namespace map

The repository layout is namespace-first, so the logical architecture should match real Kubernetes namespace boundaries. Dedicated controller namespaces are kept where they improve RBAC, upgrades, and recovery. The data namespace intentionally groups CockroachDB, Qdrant, Redis, and NATS as the shared persistence/messaging plane.

Namespace	Applications	Node pool	PSA	Mesh
kube-system	CoreDNS; NodeLocal DNSCache; Metrics Server; Proxmox CCM; Descheduler; CSI Snapshot Controller; Cilium/Hubble system components	Control plane + platform + all nodes as component requires	Privileged/system exceptions	No
istio-system	istiod; Istio CNI; ztunnel	Platform + all workers	Restricted + narrow exceptions	No
edge-system	Private Istio Gateway; cloudflared x3	Platform	Restricted	Gateway only
flux-system	Flux controllers; Flagger	Platform	Restricted	No
kyverno	Kyverno admission controllers	Platform	Restricted + controller exceptions	No
cert-manager	cert-manager	Platform	Restricted	No
openbao	OpenBao Raft cluster	Platform	Restricted	No
external-secrets	External Secrets Operator	Platform	Restricted	No
security	Falco; Falcosidekick; Kubescape; Trivy Operator; optional Tetragon	Platform + DaemonSets	Documented privileged exceptions	No
observability	Prometheus; Alertmanager; Grafana; Loki; Tempo; Pyroscope; OTel; Alloy/Faro; optional Beyla; exporters	Platform + DaemonSets	Restricted + collector exceptions	Optional
longhorn-system	Longhorn manager/engine/CSI	Workers	Privileged storage exceptions	No
velero	Velero + CSI integration	Platform	Restricted + backup permissions	No
data	CockroachDB; Qdrant; Redis; NATS	Application/data workers	Restricted	Selected service mTLS only
registry	Harbor	Platform	Restricted	No
headlamp	Headlamp	Platform	Restricted	No
aerealith-dev	Development Aerealith apps	Application	Restricted	Ambient
aerealith-staging	Staging Aerealith apps	Application	Restricted	Ambient
aerealith-prod	Production Aerealith apps	Application	Restricted	Ambient
preview-pr-*	Ephemeral pull-request apps	Application	Restricted + quota	Ambient where useful

Core Kubernetes add-on operating rules

- Proxmox cloud-controller-manager runs in HA with leader election and a dedicated, least-privilege Proxmox API token. If Talos is configured for an external cloud provider, new nodes remain unschedulable until the CCM initializes provider identity/topology; therefore monitor CCM availability as a Tier-1 cluster dependency.
- Metrics Server is for the Kubernetes resource Metrics API and autoscaling, not for observability retention. Prometheus continues scraping kubelet/resource metrics directly for monitoring.
- NodeLocal DNSCache runs as a DaemonSet and forwards cache misses to CoreDNS. Alert on local DNS pod absence, upstream failures, latency, and SERVFAIL/NXDOMAIN anomalies.
- Descheduler begins in a conservative scheduled mode. Exclude or tightly constrain CockroachDB, Qdrant, Redis, NATS, OpenBao, Longhorn, gateways, and other quorum/stateful workloads until each eviction policy is proven safe.
- Snapshot Controller is deployed with leader election and the version-matched VolumeSnapshot CRDs. Longhorn CSI handles provider-side snapshot operations; Velero and native application backups remain the independent restore layers.

12. Environment and preview namespace model

Aerealith uses one Kubernetes cluster. Environment separation is therefore enforced through namespaces, identities, network policy, quotas, and separate logical data boundaries rather than separate clusters. Production remains the highest-trust zone.

Environment	Namespace	Purpose	Data rule	Lifecycle
Development	aerealith-dev	Continuous integration, feature testing, internal development	Dedicated dev DB/schema, Qdrant collections, Redis, NATS account; synthetic/test data	Long-lived
Staging	aerealith-staging	Production-like validation, migrations, Flagger tests	Separate staging data; never prod credentials; masked fixtures if prod-like shape required	Long-lived
Production	aerealith-prod	Customer-facing production workloads	Production-only identities/data; strictest policies and approvals	Long-lived
Pull request	preview-pr-	Ephemeral UI/API/service preview for changed apps	Disposable/seeding only; never direct production database or OpenBao role	Create on PR; destroy on merge/close

Preview controls

- Each preview namespace receives ResourceQuota, LimitRange, default-deny Cilium policy, PSA Restricted labels, a unique service account set, and an expiry annotation.
- GitHub automation writes preview manifests to Git; Flux remains the cluster reconciler. CI does not receive a production kubeconfig merely to create preview pods.
- Preview deployments build only Nx-affected applications when practical. Shared dependencies are referenced through approved non-production endpoints.
- Preview secrets are issued from a dedicated OpenBao role with short TTLs and cannot read production secret paths.
- A cleanup job/controller detects stale preview namespaces and creates a Git cleanup change; emergency garbage collection is documented if Git automation fails.

Single-cluster limitation

Namespaces reduce blast radius but do not create a separate kernel or control plane. Treat previews and development as less-trusted tenants: no privileged pods, no hostPath, no host networking, strict quotas, and no production data paths.

13. Default-deny network policy model

Every workload namespace starts with deny-all ingress and egress. Allow rules ship in the same pull request as the consuming workload. Network location is not identity; Cilium rules control reachability while Istio/OpenBao/database identities authorize the operation.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: aerealith-prod
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
```

Source identity	Allowed destination	Reason
cloudflared	Private Istio Gateway only	Public origin path
Istio Gateway	Declared HTTPRoutes only	No broad namespace wildcard
frontend/BFF	Approved API services only	No direct database/secret-store access
core service SA	Exact CockroachDB/OpenBao/Redis/NATS/Qdrant endpoints required by service	Least-service reachability
async worker SA	Queue/data/object store + exact external APIs	Job-specific
Prometheus	Metrics endpoints only	Read-only scrape
OTel/Alloy	Telemetry receivers/backends only	Telemetry path
Flux	Git/OCI/DNS only	No arbitrary Internet
preview workloads	Non-production data endpoints + exact APIs	No production paths
all other	None	Default deny

14. Identity, RBAC, and administrative access

Human and machine identities remain separate. Talos API certificates are infrastructure-root credentials; Kubernetes user identities are not reused for Talos. Routine Kubernetes administration should use OIDC with phishing-resistant MFA when the self-hosted identity provider is ready.

Principal	Authentication	Authorization	Review
Talos operator	Talos mTLS certificate + VPN	Talos role certificate	Quarterly and every staff change
Kubernetes admin	OIDC + FIDO2/MFA	RBAC groups; no daily cluster-admin	Quarterly
Break-glass	Offline credential	Time-bounded cluster-admin	Every use; rotate afterward
Flux	ServiceAccount	Only reconciled API groups/namespaces	Each controller upgrade
Aerealith services	Dedicated SA + Istio identity	Kubernetes API only when required	Per release + quarterly
Data services	Dedicated service accounts + TLS identities	Namespace/network/data-layer roles	Per release + quarterly
Prometheus/Kubescape	Dedicated SA	Read-only discovery; no Secret reads	Quarterly
Headlamp users	OIDC identity	Same RBAC as kubectl; no shared token	Quarterly

- Set `automountServiceAccountToken: false` by default and use projected bound tokens with explicit audience when API access is required.
- Do not grant wildcard verbs/resources to application namespaces.
- Do not use long-lived cluster-admin ServiceAccount tokens for Headlamp.
- Generate an access export and reviewer sign-off every quarter.
- Protect GitHub, Cloudflare, Proxmox, OpenBao, backup systems, and the identity provider with hardware-backed MFA where supported.

15. Pod Security and Kyverno admission policy

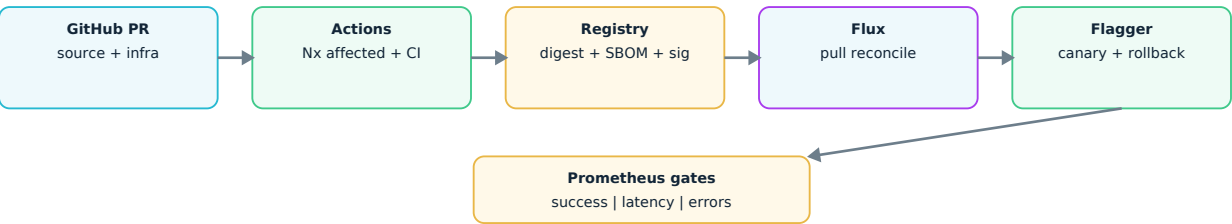
Pod Security Admission provides the non-negotiable baseline. Kyverno adds organizational policy, image verification, required metadata, resilience rules, namespace readiness, preview limits, and exception governance. Introduce policy in Audit, remediate, then enforce.

Control	Production rule	Exception
Privileged	Deny	Only named system components with owner + expiry
Root	runAsNonRoot=true	Rebuild image
Privilege escalation	false	Rare system-only exception
Capabilities	Drop ALL	Minimal named capability only
Seccomp	RuntimeDefault	No unconfined app pods
Read-only root FS	Required where supported	Document write path and compensating mount
Host namespaces	Deny hostPID/hostIPC/hostNetwork	System DaemonSet only
hostPath	Deny arbitrary	Exact path + owner + expiry
Images	Digest required in production	None
Registry	Approved registries only	Security approval
Signature	Cosign verification	Emergency incident/change exception
Resources	Requests/limits required	Calibrated system DaemonSet
Resilience	Critical service ≥ 3 replicas + PDB + spread	Approved service-tier exception
Namespace readiness	Owner + data class + PSA + quota + default deny + mesh decision	None for production
Preview expiry	Required TTL/PR metadata and quotas	No permanent preview namespace

16. Software supply-chain security

Production consumes immutable, verifiable artifacts. CI is a builder and evidence producer, not a deployment authority. Build once, promote by digest, and let Flagger/Istio prove the release before full traffic.

Stage	Required control
Source	Protected master; PR; CODEOWNERS for infra/security; dependency lockfiles; reviewed workflow changes
CI runner	Ephemeral where possible; minimal token permissions; pinned Actions by commit SHA
Build	Multi-stage/rootless image where possible; deterministic lockfile; no secrets in layers
Scan	Trivy filesystem/config/dependency/final-image scan; secret scan
SBOM	SPDX or CycloneDX attached to the digest
Provenance	SLSA-style provenance/attestation where practical
Sign	Cosign signature bound to immutable digest
Registry	Harbor target; GHCR bootstrap/fallback; audit + retention; pull-only runtime credentials
Admission	Kyverno verifies digest + signature + identity
Promotion	Git PR changes tested digest/config only; Flux reconciles
Preview	Preview images are still scanned and digest-pinned; may use shorter retention



Do not force AlwaysPullImages automatically
Digest pinning, signature verification, immutable retention, and node caches can provide stronger availability than forcing every restart to reach the registry. Revisit this only if the multi-tenant image-cache threat outweighs registry outage risk.

17. Self-hosted OCI registry strategy

Harbor remains the preferred long-term self-hosted registry. Do not make cluster bootstrap depend on a complex registry stack before storage, secrets, backup, and restore are proven.

- Bootstrap: keep GHCR as an explicit development/bootstrap exception while the cluster is built.
- Mirror: deploy Harbor after Longhorn, OpenBao, and Velero are working; mirror production images, signatures, and SBOMs locally.
- Cutover: switch production references to Harbor only after registry outage and restore behavior are tested.
- Recovery: export the minimum Tier-0/Tier-1 OCI image set to offline/PBS storage so an empty cluster can be rebuilt even if Harbor is unavailable.

Harbor concern	Design response
Registry availability	Not in normal request path, but required for reschedule/scale; monitor and cache
Database/Redis dependencies	Use supported HA design and include in Velero/native backup
Blob storage	Durable persistent/object storage with off-cluster backup
Scanning	Harbor/Trivy integration; CI scan remains independent
Trust	Cosign admission verification, not registry reputation alone
Backups	Configuration, database, blobs, signing metadata; quarterly restore

18. Secrets management and PKI

Use distinct secret layers with different recovery characteristics: offline root/recovery material, SOPS + age for bootstrap, OpenBao for runtime secrets and internal PKI, External Secrets Operator for Kubernetes-native synchronization where a standard Secret is required, and cert-manager for certificate lifecycle.

Secret class	Authority	Examples
Offline recovery	Encrypted physical custody	Talos CA recovery, OpenBao recovery, root CA, production age recovery
Bootstrap	SOPS + age	Flux bootstrap, initial OpenBao configuration, tunnel seed credentials
Runtime dynamic	OpenBao	Database users, API tokens, short-lived credentials
Kubernetes sync	External Secrets Operator	Reconcile approved OpenBao values into namespaced Kubernetes Secrets
Service certificates	OpenBao intermediate + cert-manager	Internal TLS, origin certs, monitoring clients
Mesh identity	Istio CA anchored to approved trust chain	SPIFFE-style workload identities and mTLS

- OpenBao: 3 Raft replicas, one per platform worker/PVE zone; PDB minAvailable=2.
- OpenBao is not initially meshed; use its own TLS/auth boundary and Cilium reachability controls.
- Use Kubernetes auth roles bound to namespace + service account + audience.
- External Secrets Operator receives read-only OpenBao roles scoped to exact paths and target namespaces; avoid cluster-wide secret-reader identities.
- Keep root CA offline. OpenBao holds only an online intermediate used for issuance.
- Record auth-method, policy, root/recovery, seal, and secret-engine operations as high-priority audit telemetry.

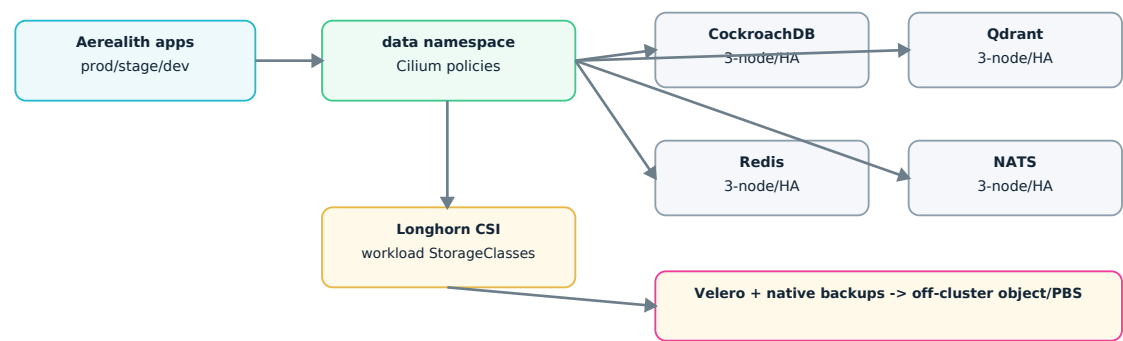
19. Storage and persistence strategy

v4.0 adopts Longhorn as the Kubernetes CSI storage layer. The important design rule is not to treat one replication factor as correct for every workload. Several selected data systems already replicate at the application layer, so storage policy must avoid unnecessary write amplification while still preserving recoverability.

Storage class / layer	Decision	Use
Proxmox local mirrored ZFS	Selected	Device redundancy, checksums, VM-disk foundation
Longhorn general-ha	Selected	General stateful services that do not provide their own strong replication; typically RF=2/3 after testing
Longhorn replicated-app	Selected	CockroachDB/OpenBao/NATS/Qdrant style systems; use a deliberately lower Longhorn RF after failure testing to avoid blind double replication
Longhorn snapshots	Convenience only	Fast rollback/snapshot workflow; not an independent backup
Ceph	Not initially	Operational complexity and three-node maintenance margin not justified
Single NFS/NAS	Not Tier-0	Creates a storage SPOF unless independently HA
Off-cluster object storage	Required for backup targets	Velero/native snapshots/Tempo/Loki or other S3 consumers as designed

Database storage rule
CockroachDB quorum is the primary availability mechanism. Do not assume Longhorn RF=3 under CockroachDB RF=3 is automatically safer. Benchmark write latency and validate node-loss/rebuild behavior, then pin a dedicated StorageClass. Native CockroachDB backups remain authoritative for database recovery.

20. Data platform: CockroachDB, Qdrant, Redis, and NATS



All four services run in the data namespace and are self-hosted. They are not interchangeable: CockroachDB is the system of record, Qdrant stores vector/search state, Redis is cache/ephemeral coordination unless explicitly promoted to durable state, and NATS is the messaging/event plane.

Service	Production topology	Primary purpose	Security / persistence rule
CockroachDB	Current CockroachDB Operator; 3 nodes; one replica per physical zone	Relational system of record, entities, permissions, transactions	TLS/cert-manager; dedicated identities; native scheduled backups; anti-affinity; no public SQL/UI
Qdrant	3-node distributed deployment; collection replicas/shards sized to workload	Vector embeddings, semantic search, retrieval indexes	API key/TLS; collection-level isolation; Qdrant snapshots exported off cluster
Redis	HA topology appropriate to cache criticality; replicas/Sentinel or cluster if required	Cache, rate limits, ephemeral coordination	Treat as disposable unless a feature explicitly requires durability; ACL/TLS; redis_exporter
NATS	3-node NATS cluster; JetStream replicas where persistence is needed	Events, queues, request/reply, async work	Accounts/users/creds; TLS; JetStream replication; nats-prometheus-exporter/native monitoring

Environment isolation inside the shared data plane

- Use separate CockroachDB databases/users/roles for dev, staging, and production; previews receive disposable databases or schemas only.
- Use separate Qdrant collections and credentials per environment; do not let preview keys enumerate production collections.
- Prefer separate Redis instances for production vs non-production when memory permits; otherwise enforce dedicated credentials/namespaces and never rely on DB index numbers as a security boundary.
- Use NATS accounts/subjects/permissions to isolate environments and services.
- Cilium policies restrict each service account to only the exact data endpoints it requires.

21. Backup and disaster recovery

Backups are layered: Velero protects Kubernetes resources and supported persistent-volume state; native application snapshots protect quorum databases and message/data systems; Talos etcd snapshots protect cluster state; physical PBS protects the hypervisor layer; and an offsite/offline copy protects against site loss and ransomware.

Asset	Authoritative backup	Initial target	Restore proof
etcd	talosctl etcd snapshot	Every 4-6h + before control-plane change	Quarterly empty-cluster recovery
Talos configs/secrets	Encrypted recovery bundle	Every approved change	Quarterly node rebuild
Kubernetes objects	Velero backup	Daily + before major change; retained off cluster	Monthly namespace and quarterly cluster restore
Longhorn PVCs	Velero CSI/FileSystem backup where supported + app-native backup	By data class; off-cluster target	Monthly representative PVC restore
CockroachDB	Native scheduled backup	<=15m incremental target + daily/full policy	Monthly isolated DB restore
Qdrant	Qdrant snapshots + metadata	Scheduled by collection criticality	Quarterly restore into isolated Qdrant
Redis	None if cache-only; RDB/AOF backup if durable	By feature requirement	Restore test only when durable
NATS JetStream	Replication + config/export/backup strategy	By stream retention/criticality	Quarterly selected-stream recovery
OpenBao	Raft snapshot	Every 4-6h + before upgrade/rekey	Quarterly new-cluster restore/unseal
Harbor	DB + blobs + config/signing metadata	Nightly/off-cluster	Quarterly registry restore
Proxmox VMs	PBS	Nightly	Monthly representative VM restore
Git repository	Remote mirror + encrypted offline bundle	Every merge	Quarterly bare-cluster bootstrap
Audit/evidence	Append-only immutable export	Hourly/daily by class	Monthly hash/query sample
PKI/recovery material	Encrypted offline custody	After key hierarchy change	Quarterly recovery ceremony

Velero is required, but not sufficient

Velero protects Kubernetes resources and can integrate with CSI/file-system backup, but it does not replace native CockroachDB/OpenBao/Qdrant recovery semantics or physical PBS. A successful backup job is not accepted until a restore path has been tested.

22. Canary and progressive delivery

Flagger remains selected because it fits the Flux operating model and integrates directly with Istio and Prometheus. A canary is treated as a deployment safety control, not merely a release convenience.

Phase	Traffic	Required evidence
Initialize	0%	Candidate pods healthy; config validation passes
Step 1	5%	Success rate, latency, dependency errors, saturation
Step 2	10%	Same + business correctness metrics
Step 3	25%	No security/audit regression; no queue/backlog growth
Step 4	50%	Sustained analysis window
Promote	100%	Automated gates pass; manual approval if high-risk
Rollback	0% candidate	Automatic if threshold exceeded; event retained

```

apiVersion: flagger.app/v1beta1
kind: Canary
metadata:
  name: aerealith-api
  namespace: aerealith-prod
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: aerealith-api
  analysis:
    interval: 1m
    threshold: 5
    maxWeight: 50
    stepWeight: 5
    metrics:
      - name: request-success-rate
        thresholdRange: { min: 99.0 }
      - name: request-duration
        thresholdRange: { max: 750 }

```

Database migration rule

Automatic traffic rollback cannot undo a destructive schema change. Production migrations follow expand -> deploy compatible code -> backfill/migrate -> verify -> contract in a later release.

23. Flux GitOps monorepo operating model

Flux is the sole production reconciler. Infrastructure lives beside application code in the Aerealith monorepo under the top-level infra directory. The Kubernetes tree is organized by actual namespace, then by application/controller inside that namespace.

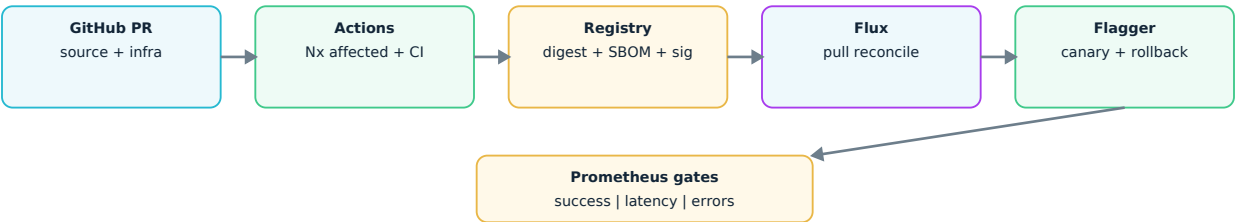
```
infra/
├── talos/
│   ├── patches/
│   └── machine-configs/      # generated secrets excluded from normal Git
├── kubernetes/
│   ├── kube-system/
│   │   ├── cilium/
│   │   └── hubble/
│   ├── istio-system/
│   ├── edge-system/
│   │   ├── istio-gateway/
│   │   └── cloudflared/
│   ├── flux-system/
│   │   ├── flux/
│   │   └── flagger/
│   ├── kyverno/
│   ├── cert-manager/
│   ├── openbao/
│   ├── external-secrets/
│   ├── security/
│   │   ├── falco/
│   │   ├── kubescape/
│   │   └── trivy-operator/
│   ├── observability/
│   │   ├── prometheus/
│   │   ├── alertmanager/
│   │   ├── grafana/
│   │   ├── loki/
│   │   ├── tempo/
│   │   ├── pyroscope/
│   │   ├── opentelemetry/
│   │   ├── alloy-faro/
│   │   └── beyla/          # optional
│   ├── longhorn-system/
│   ├── velero/
│   ├── data/
│   │   ├── cockroachdb/
│   │   ├── qdrant/
│   │   ├── redis/
│   │   └── nats/
│   ├── registry/harbor/
│   ├── headlamp/
│   ├── aerealith-dev/
│   │   ├── frontend/
│   │   ├── api/
│   │   ├── ai/
│   │   ├── workers/
│   │   ├── discord-bot/
│   │   └── integrations/
│   ├── aerealith-staging/
│   ├── aerealith-prod/
│   └── previews/          # generated preview manifests/templates
```

Reconciliation order

Order	Contents
00-bootstrap	CRDs, HelmRepositories/OCIRepositories, base namespaces
10-network	Cilium, CoreDNS/NodeLocal DNSCache, Hubble, node-encryption baseline
20-policy	PSA labels, Kyverno, default-deny templates
15-cluster-addons	Proxmox CCM, Metrics Server, Descheduler
30-storage	CSI Snapshot Controller/CRDs, Longhorn, StorageClasses, snapshots
40-secrets-pki	OpenBao, External Secrets, cert-manager issuers
50-mesh-edge	Istio, private gateway, cloudflared
60-observability-security	Prometheus/Loki/Tempo/Pyroscope/OTel/Faro + Falco/Kubescape/Trivy
70-data-registry	CockroachDB/Qdrant/Redis/NATS + Harbor
80-applications	dev, staging, production, previews
90-progressive-delivery	Flagger canaries and analysis templates
95-backup-evidence	Velero schedules, evidence export, restore/test artifacts

24. GitHub CI/CD integration

GitHub is the monorepo source of truth and CI coordinator; it is not a direct production deployment plane. GitHub Actions builds and proves artifacts, then commits or proposes the desired digest/config change. Flux pulls that desired state from Git.



Trigger	GitHub Actions responsibility	GitOps outcome
Pull request	Run Nx affected lint/typecheck/test/build; Trivy; SBOM; image build; sign; push preview image	Create/update preview manifest in Git; Flux creates preview-pr-* namespace/workloads
Merge to master	Build affected release artifacts once; retain attestations; update dev desired state	Flux reconciles aerialith-dev
Promote to staging	PR updates only image digests/config/migrations validated in dev	Flux reconciles staging; Flagger executes canary
Promote to production	Protected PR + security/change approval; same tested digest	Flux reconciles production; Flagger gates traffic
PR close/merge	Automation removes preview desired state	Flux deletes preview resources; cleanup verifies namespace removal

Repository protections

- Require CODEOWNERS approval for infra/talos, infra/kubernetes/security-sensitive paths, GitHub Actions workflows, and production overlays.
- CI tokens use least privilege; production kubeconfig and Talos credentials never live in GitHub Actions secrets.
- Pin Actions by immutable commit SHA, not floating major tags, for security-sensitive workflows.
- Use concurrency groups so two production promotions cannot race.
- Retain test, scan, SBOM, signature, canary, and reviewer evidence by release digest.

25. Cloudflare Tunnel and private ingress

Public traffic has no direct path to a node listener. Cloudflare connectors initiate outbound tunnels and reach a private ClusterIP Istio Gateway. Cloudflare Tunnel remains an explicit external edge dependency while the cluster origin stays non-public.

- One named tunnel with three cloudflared connectors, one per platform worker/PVE zone.
- Outbound tunnel traffic only, plus exact DNS/HTTPS control endpoints required by the pinned cloudflared release.
- Private Istio Gateway Service is ClusterIP only.
- No NodePort, public LoadBalancer, hostPort, or WAN 80/443 port-forward to the cluster.
- cloudflared verifies the internal origin certificate; do not use noTLSVerify.
- Management services such as Headlamp, Grafana, OpenBao, Prometheus, or Longhorn UI are not published as ordinary public application hostnames.
- If protected health information traverses Cloudflare, complete the applicable BAA, plan, subprocessor, and data-flow review before production.

Cloudflare does not create internal zero trust

Tunnel transport protects the edge connection. It does not replace Istio identity, application authentication, object-level authorization, database TLS, Cilium policy, OpenBao controls, or audit logging.

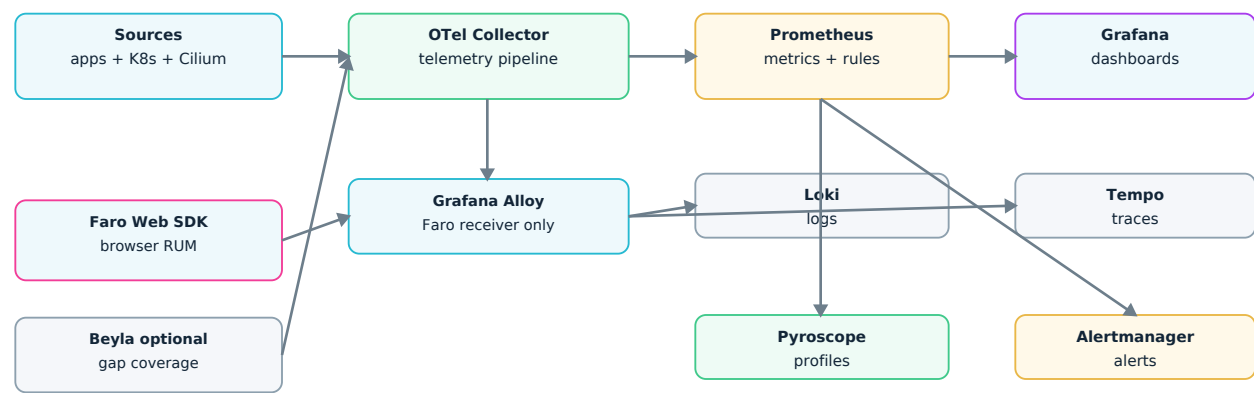
26. Runtime security and posture management

The security stack remains layered without installing duplicate engines for the same purpose. Static posture, vulnerability management, runtime detection, network telemetry, and audit each have a distinct job.

Tool	Maturity / role	Decision
Cilium/Hubble	CNCF Graduated; network policy/flow visibility	Required
Istio	CNCF Graduated; mesh identity/mTLS/L7	Required
Kyverno	CNCF project; admission/image policy	Required
Trivy / Trivy Operator	CVE, config, IaC, image scanning	Required
Kubescape	CNCF Incubating; posture/compliance scanning	Required
Falco + Falcosidekick	CNCF Graduated runtime detection + alert routing	Required after tuning
Tetragon	eBPF runtime observability/enforcement	Optional advanced phase
OPA Gatekeeper	Second admission engine	Not initially - avoid duplicate policy logic
Beyla	eBPF application auto-instrumentation	Optional observability coverage; not a security control

Falco runs initially in observation mode and its host privileges are documented Kyverno exceptions. Tetragon is only added if deeper eBPF tracing/enforcement is justified after Cilium/Hubble/Falco coverage is understood.

27. Observability and telemetry architecture



OpenTelemetry remains the primary telemetry API/SDK and backend transport for Aerealith code. Prometheus remains the metrics and alert-rule engine. The Grafana stack is self-hosted: Loki for logs, Tempo for traces, Pyroscope for continuous profiling, Grafana for visualization, Faro for browser RUM, and optional Beyla for eBPF auto-instrumentation where code instrumentation is missing or impractical.

Component	Deployment intent	HA / retention intent
Prometheus	2 independent replicas on platform workers	7-15 day local retention initially; no shared TSDB dependency
Alertmanager	3 replicas	HA mesh; minAvailable=2
OpenTelemetry Collector	DaemonSet + 2 gateway collectors	OTLP mTLS where useful; bounded queues/backpressure
Loki	Small HA topology	Operational logs; retention by data class
Tempo	Small HA topology with off-cluster/object backend as selected	Distributed traces; sampling/cardinality limits
Grafana	2 replicas if UI HA required	Private; dashboards provisioned from Git
Pyroscope	HA appropriate to ingest scale	Profiles correlated to metrics/traces; retention by cost/risk
Grafana Alloy	Narrow deployment for self-hosted Faro receiver	Not a duplicate general collector
Faro Web SDK	Frontend application library	RUM metrics/logs/errors/traces; privacy filtering before send
Beyla	Optional DaemonSet/selected nodes	Gap coverage for uninstrumented or third-party services
Hubble Relay	2 replicas	Network flow visibility; UI private/disabled by default
External Prometheus	10.42.0.20 existing sentinel	Outside-cluster view for Proxmox/PBS/network/public synthetics

Beyla decision
Because Aerealith is being instrumented directly with OpenTelemetry, Beyla is optional. Use it for blind spots, third-party binaries, rapid RED metrics, or to validate instrumentation coverage - not as a reason to remove intentional code-level spans and business metrics.

28. Exporters and scrape matrix

Prefer native Prometheus endpoints where projects expose them. Add exporters only where they provide a missing translation layer. A separate cAdvisor deployment is not required by default because kubelet exposes container/cAdvisor metrics used by Prometheus.

Source	Collection method	What it covers
Nodes	node_exporter	CPU, memory, filesystem, network, host-level saturation
Kubernetes objects	kube-state-metrics	Deployments, StatefulSets, Jobs, PVCs, quotas, pod status
Autoscaling resource metrics	Metrics Server / metrics.k8s.io	Current pod/node CPU and memory for HPA/VPA inputs and kubectl top; not long-term monitoring
Cluster DNS cache	NodeLocal DNSCache/CoreDNS metrics	Per-node DNS cache request rate, latency, upstream failures and cache behavior
Kubelet/cAdvisor	Kubelet metrics endpoints	Container CPU/memory/fs/network metrics
Cilium/Hubble	Native Cilium/Hubble metrics	Agent/operator health, policy, flows, drops, DNS
Istio/Envoy	Native Prometheus endpoints	Request rate/errors/duration, mTLS, proxy/gateway health
Flux controllers	Native metrics	Reconcile success/failure/duration, source readiness
Flagger	Native/analysis metrics	Canary status and rollout decisions
Longhorn	Native metrics	Volumes, replicas, disk capacity, rebuild health
Velero	Native metrics	Backup/restore success, duration, item failures
OpenBao	Telemetry endpoint	Seal state, request/lease/auth/system metrics
CockroachDB	Prometheus-format metrics	SQL, ranges, replicas, storage, queues, node health
Qdrant	Native metrics	Requests, collections, cluster/consensus/storage
Redis	redis_exporter or supported native integration	Memory, commands, hit ratio, replication, evictions
NATS	nats-prometheus-exporter / monitoring endpoint	Connections, routes, subscriptions, JetStream health
cloudflared	Prometheus metrics endpoint	Tunnel connector health, requests, connection errors
Harbor	Native/available metrics	Registry health, jobs, storage, scans
Faro receiver	Alloy component metrics	Ingested RUM signals, errors, payload/rate limits
Beyla	Native exported OTel/Prometheus	RED metrics and traces for selected services

29. Audit logging and evidence-quality telemetry

Audit data answers different questions at different layers. Kubernetes API audit proves administrative actions against cluster resources; Talos logs prove node lifecycle/configuration events; OpenBao audit proves secret operations; data-service audit/metrics prove privileged data operations; Aerealith application audit proves which user touched which protected business object.

Source	Minimum events	Retention handling
Kubernetes API	authn failures, RBAC changes, Secret metadata access, exec/port-forward, admission/policy changes, deployments	Searchable + immutable high-value export
Talos	config apply, upgrade, reboot, API authentication, service/kernel events	Central collector + evidence snapshots
Cilium/Hubble	denies, unusual east-west, DNS/egress anomalies	Short operational + incident export
Istio	mTLS failures, authz denies, gateway access	Operational; redact sensitive headers
OpenBao/ESO	seal/unseal, root, auth/policy, secret lease/sync, denied access	Restricted immutable audit path
CockroachDB	node/SQL security events, privileged SQL, backup operations	Data-class retention
Qdrant/Redis/NATS	admin/auth changes, cluster health, backup/restore, access failures	Operational + incident/evidence as risk requires
Flux/Kyverno/Flagger	reconciles, drift, admission denies, exceptions, rollout/rollback	Per change + monthly evidence
Velero/Longhorn	backup/restore/snapshot/delete failures and changes	Per job + evidence
Aerealith	actor, tenant, object, action, purpose, result, correlation ID	Purpose-specific protected audit store

Do not log secrets or raw protected payloads
Authorization headers, session cookies, passwords, tokens, private keys, raw prompts containing sensitive data, and full database rows must not appear in ordinary telemetry. Faro/browser telemetry requires particular care with URLs, user attributes, console logs, DOM/session data, and source maps.

30. CockroachDB security and Kubernetes placement

CockroachDB now runs inside the cluster using the current CockroachDB Kubernetes Operator. The August 2026 operator is the preferred self-hosted Kubernetes lifecycle path. The database remains logically independent through native replication, TLS, dedicated identities, native backups, and strict scheduling - even though it shares the Kubernetes failure domain.

Control	Production rule
Topology	3 CockroachDB nodes; one per Proxmox zone/worker hostname; hard anti-affinity and topology spread
Storage	Dedicated Longhorn StorageClass tuned for CockroachDB; benchmark latency; avoid blind storage RF=3 + CRDB RF=3 duplication
TLS	Secure deployment; cert-manager/OpenBao intermediate where validated; rotate automatically
Application connection	Cluster-internal service; clients use TLS and multi-node/service discovery; no public SQL listener
Identities	Separate runtime, migration, monitoring, backup, analytics, and break-glass users
Network	Cilium service-account policy; only approved Aerealith services and monitoring reach SQL/metrics
Admin UI	Private only; management identity/RBAC; never public app hostname
Backups	Native scheduled backups are authoritative; Velero protects K8s objects/PVC context but does not replace SQL backup
Upgrades	Operator-controlled rolling upgrade; backup and health preflight; one quorum member at a time
Failure objective	One worker/PVE host loss must not require manual data repair before service resumes
Recovery consequence of moving CRDB into Kubernetes The database now shares more infrastructure with the application platform. Compensate with tested etcd/Flux/Velero rebuild procedures, native CockroachDB backups stored off cluster, strict topology spread, and enough worker capacity to restore quorum after one physical-host loss.	

31. Failure-mode analysis

Failure	Expected automatic result	Manual action	Residual risk
One control-plane VM	Kubernetes API continues via 2/3 etcd; VIP re-elects	Rebuild member on same physical domain	Second CP loss
One platform worker	Istio/Flux/security/storage controllers continue on two	Repair/rebuild; replicated state rejoins	Capacity/quorum placement
One application worker	Critical apps and data quorum continue on two	Repair/rebuild; CRDB/Qdrant/NATS rebalance	N+1 request + I/O capacity
One entire Proxmox host	2 CP + 2 sys + 2 app + surviving data quorum	Stabilize; rebuild one member of each set	Shared gateway/switch/ISP/site
One cloudflared connector	Other connectors remain	Investigate connector	Cloudflare account/provider
One Istio gateway/waypoint	Replica routing continues	Replace pod; inspect config	Insufficient replicas
One CRDB node	2/3 database quorum	Operator replaces/rebalances	Second DB loss before repair
One Qdrant peer	Replicated collections continue where replica factor permits	Replace/rejoin; verify shard replication	Under-replicated collection
One Redis node	Cache/HA topology fails over if configured	Repair; accept cache miss if disposable	If Redis was incorrectly used as source of truth
One NATS node	Cluster/JetStream quorum continues if replica count allows	Replace/rejoin	Under-replicated stream
One OpenBao replica	2/3 Raft remains	Replace/rejoin	Second replica loss
One Longhorn replica/disk	Volume remains if StorageClass has surviving replica or app-level quorum covers it	Rebuild replica/volume; verify app state	Storage RF choice
Bad GitOps commit	Policy/canary may block; some bad desired state can reconcile	Revert + incident review	Git trust plane
Site power/cooling	After UPS runtime, primary site unavailable	Invoke site recovery	Single-site compute

Availability ceiling

Kubernetes can be HA while the site is not. A single gateway, aggregation switch, ISP, power/cooling domain, or physical location can still dominate availability. Cluster HA is not the same as site HA.

32. Incident containment and recovery

Incident workflow preserves evidence first, then contains with identity/network controls and rebuilds immutable components. GitOps automation is suspended only as narrowly and briefly as required to stop the controller from undoing containment.

- Declare severity and incident commander; synchronize clocks and preserve evidence.
- Freeze risky automated changes and determine whether Flux needs temporary suspension for affected resources.
- Revoke exposed identities and deny network paths with Cilium/Talos/Istio controls.
- Snapshot relevant audit/log/data evidence before destructive remediation.
- Rebuild compromised Talos nodes or workloads from known-good image/config rather than hand-repairing them.
- Restore data only from verified native/Velero/PBS backups; rotate secrets and PKI as required.
- Re-enable production traffic through canary/progressive delivery, not instant full cutover.
- Close with root cause, control failure, evidence links, owner, due date, and documented risk acceptance for deferred remediation.

Break-glass reconciliation

If an emergency requires direct kubectl mutation, record the exact diff and incident ID, then reconcile Git immediately after containment. The cluster must not remain in a permanently undocumented state.

33. Patch, upgrade, and vulnerability management

Security depends on staying within supported release windows. Treat the compatibility/version matrix as an operational control, not a one-time installation choice. v4.0 also adds data/operator/storage compatibility to the release gate.

Component	Production rule
Talos	Latest supported patch on accepted minor; control plane one node at a time; workers one failure domain at a time
Kubernetes	Upgrade one minor at a time; staging compatibility and restore test first
Cilium	Latest patch before minor upgrade; preflight/connectivity tests; validate Istio integration
Istio	Supported minor/current patch; revision-based upgrade/canary where possible
Flux/Flagger/Kyverno	Pin controller/chart versions; policy/reconcile tests in staging before production
Longhorn	Compatibility gate with Kubernetes/Talos kernel; snapshot/restore proof before major upgrade
Velero	Pin server/plugins; verify object-store and CSI compatibility; test restore after upgrade
OpenBao	Raft snapshot before upgrade; one member at a time
CockroachDB Operator/DB	Native backup first; operator/DB compatibility; rolling operator-managed upgrade
Qdrant/Redis/NATS	Backup/replication health first; one node/peer at a time where supported
Observability stack	Avoid simultaneous backend upgrades; preserve telemetry during control-plane changes
Talos Secure Boot image	Preserve signing/PCR/recovery keys and validate rollback before fleet rollout

The v3.0 compatibility example - Talos 1.13.9 / Kubernetes 1.36.3, Cilium 1.20.x, and Istio 1.30.4 as of 30 August 2026 - remains an example, not a permanent pin. Re-verify current stable versions and compatibility matrices at implementation time.

34. Vulnerability-management SLA

Severity	Target remediation	Production rule
Critical - exploited or Internet-reachable	24 hours	Emergency change/canary; isolate if patch unavailable
Critical - no known exploitation	72 hours	Patch or documented compensating control
High	7 days	Patch in maintenance window; risk exception if missed
Medium	30 days	Planned remediation
Low	90 days / backlog	Risk-based prioritization

- Scan code and images on every build and rescan running production images daily or when vulnerability databases materially update.
- Generate a daily inventory of image digests, node versions, operators, CRDs, and data-service versions.
- Track vulnerability age, not only raw count.
- Require exception owner, justification, compensating control, approval, and expiry.
- Patch Talos, Proxmox, network gear, firmware, data operators, and Kubernetes controllers as separate asset classes with their own support windows.
- Record emergency mitigations such as NetworkPolicy, route removal, or feature disablement as temporary controls, not permanent fixes.

35. Security standards and compliance mapping

Technical architecture supports control objectives, but auditors certify or attest an organization and system of controls, not a list of tools. ISO/IEC 27001 is an ISMS standard; SOC 2 is an independent attestation; HIPAA requires administrative, physical, and technical safeguards; GDPR requires risk-appropriate security and accountability.

Control domain	Technical implementation	Evidence examples	Framework relevance
Access	VPN, Talos mTLS, OIDC/RBAC, OpenBao/ESO policy	RBAC export, admin cert inventory, quarterly review	ISO, SOC 2, HIPAA, GDPR
Change/configuration	Protected GitHub, Flux, Talos config review, canary	PR approvals, config hashes, reconciliation/canary records	ISO, SOC 2, HIPAA
Network security	Talos firewall, Cilium, Istio mTLS	Firewall exports, policies, Hubble, cert inventory	All
Supply chain	Trivy, SBOM, Cosign, Kyverno, Harbor	Scan reports, SBOM, signature verification, registry audit	ISO, SOC 2
Logging/detection	K8s/Talos/OpenBao/data audit, Falco, Hubble, Faro privacy controls	Alert tests, immutable logs, incident records	All
Resilience	HA, Longhorn, Velero, native backups, PBS, restore tests	Restore reports, failover timings, checksums	ISO, SOC 2 Availability, HIPAA
Privacy lifecycle	Classification, retention, deletion, browser telemetry minimization	RoPA/DPIA/deletion tests, telemetry review	ISO 27701/GDPR
Physical/media	Rack, UPS, disk custody, sanitization	Access logs, UPS tests, asset/disposal records	ISO, HIPAA
Suppliers	Cloudflare/GitHub/provider review	BAA/DPA/subprocessor and risk records	ISO, SOC 2, HIPAA, GDPR

36. Automated compliance evidence

Every important automated control should have an owner, collection method, frequency, retention class, and review step. A dashboard is not audit evidence unless its underlying state is exported, attributable, and retained.

```
compliance/  
  controls/  
    mappings/  
      iso27001-2022.yaml  
      soc2-tsc.yaml  
      hipaa.yaml  
      gdpr.yaml  
  evidence/  
    access/ network/ policy/ supply-chain/  
    backups/ availability/ storage/ data/  
    physical/ privacy/ vendors/ incidents/ audits/
```

Evidence family	Automated artifact	Review cadence
Talos/node	Machine config hash, node version, firewall, Secure Boot/encryption	Daily snapshot; monthly review
Kubernetes access	RBAC, bindings, admin auth settings	Monthly diff; quarterly sign-off
Network	Cilium policies, Talos firewall, public route scan	Daily; monthly review
Admission	Kyverno policies, PolicyReports, exceptions	Daily; weekly exception review
Supply chain	Digest, Trivy, SBOM, Cosign verification	Per release + daily rescan
GitOps/canary	Flux revision, Flagger analysis, rollback/promotion	Per change
Storage/backup	Longhorn health, Velero jobs, native backup checksums, restore results	Per job; monthly/quarterly
Data	CRDB/Qdrant/Redis/NATS topology, auth and backup state	Daily health; monthly control review
Detection	Audit health, Falco rules, alert test results	Continuous + monthly test
Observability privacy	Faro/OTel field allowlists, sampling, redaction, source-map handling	Per release + quarterly
Physical	UPS/sensors/assets/media custody	Monthly/quarterly

Evidence minimization

Do not copy raw PHI, customer payloads, raw prompts, browser replay data, or secrets into the compliance repository. Store sensitive evidence in a restricted immutable store and commit only manifest, hash, time window, control ID, and reviewer result.

37. Phased implementation plan

Phase	Outcome	Hard exit gate
0 - Inventory	Exact hosts, IPs, NICs, disks, power, routes, data classes	No invented IP/capacity values
1 - Proxmox foundation	Patched hosts, ZFS, VM firmware, firewall templates, PBS console path	Quorum + backup console tested
2 - Talos control plane	3 CP nodes + etcd + VIP + default-block host firewall	Kill one CP; API/etcd remain healthy
3 - Platform workers	3 sys workers with topology labels/taints	Loss of one sys worker does not remove critical platform
4 - Application workers	3 app workers + N+1 capacity proof	Loss of one app worker within SLO
5 - Cilium	CNI, WireGuard, Hubble, default-deny baseline	Connectivity test + encryption proof
6 - Core add-ons	Proxmox CCM, Metrics Server, NodeLocal DNSCache, Descheduler	Node initialization/topology, metrics API, DNS failover/cache, and safe dry-run/deschedule policy proven
7 - Istio Ambient	istiod/ztunnel/gateway, strict mTLS in app namespaces	mTLS/authz test suite passes
8 - Flux/SOPS/Kyverno	GitOps authoritative, policies enforced	Direct prod deploy path disabled/documented
9 - Longhorn/snapshots	CSI Snapshot Controller/CRDs, Longhorn, StorageClasses, topology, snapshot policy	VolumeSnapshot create/restore plus volume failure/rebuild and node loss proven
10 - Secrets/PKI	OpenBao + ESO + cert-manager + rotation	Raft snapshot restore and secret rotation succeeds
11 - Observability/security	Prometheus/Loki/Tempo/Pyroscope/OTel/Faro + Falco/Kubescape/Trivy	Detection, trace/log/profile and alert tests succeed
12 - Data plane	CockroachDB + Qdrant + Redis + NATS	One-host loss preserves required quorum and app service
13 - Ingress/canary	cloudflared x3 + Flagger	Kill connector + force failed canary rollback
14 - Velero/backups	Velero + native data backups + PBS/offsite	Namespace/PVC/data/etcd/OpenBao restore proofs
15 - Environments/previews	dev/staging/prod + automated PR preview lifecycle	Preview create/delete and isolation tests pass
16 - Aerealith production	Digest-pinned workloads, SLOs, data policies	One-host failure exercise succeeds
17 - Evidence/DR	Automated evidence + quarterly runbooks	Signed readiness review

Implementation rule

Do not install the entire toolchain first and harden it afterward. Each phase closes with a failure test and evidence package before the next phase becomes production dependency.

38. Production validation checklist

Cluster foundation

- Three control-plane nodes are on three different physical hosts and normal workloads cannot schedule there.
- Talos API is reachable only from approved management sources; Kubernetes API VIP fails over and is not public.
- etcd quorum, snapshot, and restore have been tested; Talos ingress firewall defaults to block.

Core Kubernetes add-ons

- Proxmox cloud-controller-manager is healthy with leader election; node providerID/topology labels are correct; its Proxmox API token is least privilege and stored through OpenBao/ESO.
- Metrics Server serves metrics.k8s.io and kubectl top/HPA tests succeed without weakening kubelet TLS verification.
- NodeLocal DNSCache is present on intended nodes, forwards to CoreDNS, and continues resolving during a CoreDNS pod loss within the tested resilience envelope.
- Descheduler policy is version-controlled, PDB-aware, and excludes unproven quorum/stateful workloads; an intentional placement-drift test is corrected without availability loss.
- CSI Snapshot Controller and VolumeSnapshot CRDs are healthy; a Longhorn-backed VolumeSnapshot can be created and restored; snapshots are not treated as the only backup.

Network, mesh, and environments

- Cilium connectivity test passes; WireGuard encryption is verified cross-node; default-deny exists in every app/preview namespace.
- Istio strict mTLS is verified for selected namespaces; Cilium and Istio L7 responsibilities do not overlap.
- No public NodePort/LoadBalancer/hostPort exists; preview namespaces cannot reach production data or secrets.

Workload and supply-chain security

- PSA Restricted is enforced on application namespaces and Kyverno is Enforce with documented exceptions only.
- All production images are digest-pinned and signed; Trivy/Kubescape scans have no unaccepted Critical/High finding.
- Falco alerts reach the incident channel; service accounts are dedicated; token automount is off unless needed.

Data, storage, and recovery

- CockroachDB, Qdrant, Redis, NATS, OpenBao, and Longhorn are topology-spread and have explicit PDB/replication policies.
- One complete application-worker/PVE-host loss preserves required data and service quorum.
- Velero restore, CockroachDB restore, Qdrant restore, OpenBao restore, etcd recovery, and representative PBS restore are documented and measured.
- Longhorn snapshots are not being counted as the only backup for any Tier-0 data.

Observability and operations

- Prometheus/Alertmanager/Loki/Tempo/Pyroscope/Grafana health alerts work; OTel traces correlate with logs/metrics.
- Faro RUM is privacy-reviewed and can correlate frontend to backend traces without sending forbidden data.
- Beyla is either disabled or explicitly scoped to services where it provides coverage value.
- Headlamp is private, uses OIDC/RBAC, and has no shared cluster-admin token.

39. Definitive architecture

The architecture I would build

Talos Linux + upstream Kubernetes on three dedicated control-plane VMs, three platform workers, and three application workers. Keep normal workloads off the control plane. Use Cilium on every node for L3/L4 policy, Hubble, and WireGuard; Istio Ambient for application workload identity, mTLS, L7 authorization, and traffic shifting; Flux as the only production reconciler; and Flagger for Prometheus-gated canaries.

Run the Proxmox cloud-controller-manager, Metrics Server, NodeLocal DNSCache, Descheduler, and CSI Snapshot Controller as core Kubernetes add-ons. Run Longhorn as the CSI layer with workload-specific StorageClasses; OpenBao + External Secrets Operator + cert-manager for secrets/PKI; Kyverno for admission; Trivy, Kubescape, and Falco for security; OpenTelemetry + Prometheus + Loki + Tempo + Grafana + Pyroscope + Faro for observability; optional Beyla for instrumentation gaps; Headlamp as the private operations UI; Harbor as the self-hosted registry target; and Velero + native snapshots + physical PBS + offsite/offline copies as layered recovery.

Run CockroachDB, Qdrant, Redis, and NATS in the data namespace. CockroachDB is the system of record, Qdrant is the vector store, Redis is cache/ephemeral coordination unless explicitly designed as durable state, and NATS is the event/message plane. Spread every quorum system across physical Proxmox zones and never allow preview environments to use production data credentials.

The GitHub monorepo contains infra/talos and infra/kubernetes. The Kubernetes tree is namespace-first and app-second. GitHub Actions uses Nx affected to build/test/scan/sign images, then changes desired state in Git. Flux pulls the change, and Flagger/Istio prove production releases. Development, staging, production, and preview-pr-* all live in the same cluster but have explicit namespace, RBAC, network, quota, secret, and data boundaries.

Keep	Replace / change from v3.0
Three physical Proxmox failure domains	Add Proxmox CCM + Metrics Server + NodeLocal DNSCache + Descheduler + CSI Snapshot Controller
Talos Linux + upstream Kubernetes	CockroachDB dedicated VMs -> CockroachDB Operator inside Kubernetes
Cilium + Hubble + WireGuard	Local-only PV strategy -> Longhorn CSI with workload-specific StorageClasses
Cilium + Hubble + WireGuard	Optional Velero -> required Velero backup layer
Istio Ambient	Jaeger -> Tempo
Flux + Flagger	Basic observability -> Pyroscope + Faro + optional Beyla + exporter matrix
Cloudflare Tunnel	No operator UI -> private Headlamp
OpenBao + cert-manager	OpenBao-only sync -> add External Secrets Operator
Harbor target	Separate GitOps repo example -> monorepo infra/talos + infra/kubernetes
Falco + Kubescape + Trivy	Functional app namespaces only -> dev/staging/prod/preview environment model
PBS + offsite recovery	External CRDB firewall paths -> data services ClusterIP + Cilium identity controls

A. Configuration baselines

Control-plane VIP document

```
apiVersion: v1alpha1
kind: Layer2VIPConfig
name: 10.42.0.TBD
link: <CONTROL_PLANE_INTERFACE>
```

Platform worker labels and taints

```
machine:
  nodeLabels:
    aerealith.io/node-pool: platform
    topology.kubernetes.io/zone: pve-a
  nodeTaints:
    aerealith.io/node-pool: platform:NoSchedule
```

Application worker labels

```
machine:
  nodeLabels:
    aerealith.io/node-pool: application
    topology.kubernetes.io/zone: pve-a
```

Critical workload spread

```
topologySpreadConstraints:
- maxSkew: 1
  topologyKey: topology.kubernetes.io/zone
  whenUnsatisfiable: DoNotSchedule
  labelSelector:
    matchLabels:
      app.kubernetes.io/name: aerealith-api
- maxSkew: 1
  topologyKey: kubernetes.io/hostname
  whenUnsatisfiable: DoNotSchedule
  labelSelector:
    matchLabels:
      app.kubernetes.io/name: aerealith-api
```

Environment namespace baseline

```
apiVersion: v1
kind: Namespace
metadata:
  name: aerealith-prod
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/audit: restricted
    pod-security.kubernetes.io/warn: restricted
    istio.io/dataplane-mode: ambient
    aerealith.io/environment: production
    aerealith.io/data-class: confidential
```

B. Port and firewall matrix

Source	Destination	Port/protocol	Purpose	Default
Management VPN	Talos nodes	50000/TCP	Talos API	Allow named subnet only
Management VPN + nodes	K8s API VIP/CP	6443/TCP	Kubernetes API	Allow
Control planes	Control planes	2379-2380/TCP	etcd	CP /32 only
Control plane / monitoring	Nodes	10250/TCP	Kubelet	Approved sources
K8s nodes	K8s nodes	8472/UDP	Cilium VXLAN	Node set only
K8s nodes	K8s nodes	51871/UDP	Cilium WireGuard	Node set only
Workers	Workers	15008/TCP	Istio HBONE	Worker set only
Workers	Workers	Longhorn required ports	CSI/replica/engine traffic	Worker set only; pin from docs
cloudflared	Cloudflare	7844/TCP+UDP	Tunnel	Outbound only
Aerealith services	CockroachDB ClusterIP	26257/TCP TLS	SQL	Exact service accounts only
Monitoring	Data/infra metrics endpoints	Service-specific	Prometheus scrape	Monitoring identity only
Browsers	Faro receiver route	443 via approved app telemetry route	Frontend telemetry	CORS/rate/auth policy
Flux/nodes	Git/OCI	443/TCP	Git/image pull	Approved endpoints
All systems	DNS/NTP	53 TCP/UDP, 123 UDP	Resolution/time	Approved services
PVE	PBS	8007/TCP	Backup	Backup clients only
Ordinary LAN	K8s/PVE/data/OpenBao	Any	No business need	DROP

C. Component maturity and selection

CNCF maturity is a useful selection signal, not a substitute for threat modeling, supported-version policy, security advisories, or internal operational competence. The table also distinguishes non-CNCF ecosystem components that are selected because they fill a concrete requirement.

Project	Maturity / foundation	Use in Aerealith
cloud-controller-manager / Proxmox CCM	Kubernetes control-plane extension + Proxmox provider project	Node identity/lifecycle and Proxmox topology labels
Metrics Server	Kubernetes SIG Instrumentation	Resource Metrics API for autoscaling and kubectl top
NodeLocal DNSCache	Kubernetes stable cluster add-on	Per-node DNS caching in front of CoreDNS
Descheduler	Kubernetes SIG Scheduling	Policy-driven correction of placement drift
CSI Snapshot Controller	Kubernetes SIG Storage / CSI external-snapshotter	Cluster-wide VolumeSnapshot lifecycle
Kubernetes	CNCF Graduated	Orchestration
Cilium	CNCF Graduated	CNI/network security/Hubble
Istio	CNCF Graduated	Ambient service mesh
Flux	CNCF Graduated	GitOps
Prometheus	CNCF Graduated	Metrics/alert rules
cert-manager	CNCF Graduated	Certificate automation
Harbor	CNCF Graduated	Self-hosted registry
Falco	CNCF Graduated	Runtime detection
OpenTelemetry	CNCF Graduated	Telemetry APIs/collectors
NATS	CNCF Incubating	Messaging/event plane
Longhorn	CNCF Incubating	Kubernetes persistent storage
Kubescape	CNCF Incubating	Posture/compliance scanning
Velero	CNCF Sandbox (2026)	Kubernetes backup/restore/migration
external-secrets	CNCF Sandbox	OpenBao-to-Kubernetes secret reconciliation
Headlamp	CNCF Sandbox / Kubernetes SIG UI	Kubernetes operations UI
Kyverno	CNCF project	Admission and image verification
Flagger	Flux ecosystem	Progressive delivery
OpenBao	Linux Foundation ecosystem	Runtime secrets/PKI
Talos Linux	Sidero Labs ecosystem	Secure immutable node OS
CockroachDB Operator	Cockroach Labs	Distributed SQL lifecycle
Qdrant	Qdrant OSS	Vector database
Redis	Redis ecosystem	Cache/ephemeral data
Grafana Loki	Grafana OSS	Logs
Grafana Tempo	Grafana OSS	Traces
Grafana Pyroscope	Grafana OSS	Continuous profiling
Grafana Faro	Grafana OSS	Frontend/RUM telemetry
Grafana Alloy	Grafana OSS	Self-hosted Faro receiver only
Grafana Beyla	Grafana OSS	Optional eBPF auto-instrumentation
Grafana	Grafana OSS	Visualization

D. Authoritative references

ID	Reference	URL
R1	Talos Security Checklist	https://docs.siderolabs.com/talos/v1.12/security/talos-security-checklist
R2	Talos default hardening and CIS compliance	https://docs.siderolabs.com/talos/v1.12/security/talos-default-hardening-and-cis-compliance
R3	Talos virtual shared IP	https://docs.siderolabs.com/talos/v1.13/networking/advanced/vip
R4	Talos Cilium deployment guide	https://docs.siderolabs.com/kubernetes-guides/cni/deploying-cilium
R5	Talos ingress firewall	https://docs.siderolabs.com/talos/v1.12/networking/ingress-firewall
R6	Talos disaster recovery	https://docs.siderolabs.com/talos/v1.10/build-and-extend-talos/cluster-operations-and-main-tenance/disaster-recovery
R7	Cilium and Istio integration	https://docs.cilium.io/en/latest/network/servicemesh/istio/
R8	Cilium Kubernetes compatibility	https://docs.cilium.io/en/stable/network/kubernetes/compatibility/
R9	Istio Ambient platform prerequisites	https://istio.io/latest/docs/ambient/install/platform-prerequisites/
R10	Istio 1.30 release line	https://istio.io/latest/news/releases/1.30.x/
R11	Istio supported releases	https://istio.io/latest/docs/releases/supported-releases/
R12	Flux SOPS guide	https://fluxcd.io/flux/guides/mozilla-sops/
R13	Flagger Istio progressive delivery	https://docs.flagger.app/main/tutorials/istio-progressive-delivery
R14	Kyverno documentation	https://kyverno.io/docs/
R15	Falco Kubernetes deployment	https://falco.org/docs/setup/kubernetes/
R16	Kubescape CNCF project	https://www.cncf.io/projects/kubescape/
R17	Harbor CNCF project	https://www.cncf.io/projects/harbor/
R18	cert-manager CNCF project	https://www.cncf.io/projects/cert-manager/
R19	OpenTelemetry CNCF project	https://www.cncf.io/projects/opentelemetry/
R20	SPIFFE CNCF project	https://www.cncf.io/projects/spiffe/
R21	Kubernetes HA with kubeadm	https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/high-availability/
R22	RKE2 CIS hardening guide	https://docs.rke2.io/security/hardening_guide
R23	CockroachDB certificates	https://www.cockroachlabs.com/docs/stable/create-security-certificates-openssl
R24	CockroachDB backup monitoring	https://www.cockroachlabs.com/docs/stable/backup-and-restore-monitoring
R25	CNCF project maturity model	https://contribute.cncf.io/projects/lifecycle/
R26	HIPAA Security Rule	https://www.hhs.gov/hipaa/for-professionals/security/index.html
R27	GDPR Article 32	https://eur-lex.europa.eu/eli/reg/2016/679/art_32/oj/eng
R28	ISO/IEC 27001 overview	https://www.iso.org/isoiec-27001-information-security.html
R29	AICPA SOC 2 overview	https://www.aicpa-cima.com/topic/audit-assurance/audit-and-assurance-greater-than-soc-2
R30	Cloudflare Tunnel HA replicas	https://developers.cloudflare.com/cloudflare-one/networks/connectors/cloudflare-tunnel/configure-tunnels/tunnel-availability/deploy-replicas/
R31	Proxmox Backup Server features	https://www.proxmox.com/en/products/proxmox-backup-server/features
R32	Talos logging	https://docs.siderolabs.com/talos/v1.13/configure-your-talos-cluster/logging-and-telemetry/logging
R33	Talos Secure Boot	https://docs.siderolabs.com/talos/v1.13/platform-specific-installations/bare-metal-platforms/secureboot
R34	Talos disk encryption	https://docs.siderolabs.com/talos/v1.12/configure-your-talos-cluster/storage-and-disk-management/disk-encryption
R35	Cilium system requirements	https://docs.cilium.io/en/stable/operations/system_requirements/
R36	Tetragon Kubernetes installation	https://tetragon.io/docs/installation/kubernetes/
R37	CNCF projects directory	https://www.cncf.io/projects/
R38	Longhorn CNCF project	https://www.cncf.io/projects/longhorn/
R39	Velero documentation	https://velero.io/docs/v1.18/

ID	Reference	URL
R40	CockroachDB Kubernetes Operator GA announcement	https://www.cockroachlabs.com/blog/cockroachdb-kubernetes-operator/
R41	CockroachDB Kubernetes performance guidance	https://www.cockroachlabs.com/docs/v26.1/kubernetes-performance
R42	Qdrant distributed deployment	https://qdrant.tech/documentation/scaling/distributed_deployment/
R43	NATS CNCF project	https://www.cncf.io/projects/nats/
R44	External Secrets CNCF project	https://www.cncf.io/projects/external-secrets/
R45	Grafana Tempo documentation	https://grafana.com/docs/tempo/latest/
R46	Grafana Pyroscope documentation	https://grafana.com/docs/pyroscope/latest/
R47	Grafana Faro OSS	https://grafana.com/oss/faro/
R48	Grafana Alloy Faro receiver	https://grafana.com/docs/alloy/latest/reference/components/faro/faro.receiver/
R49	Grafana Beyla documentation	https://grafana.com/docs/beyla/latest/
R50	Headlamp CNCF project	https://www.cncf.io/projects/headlamp/
R51	Headlamp in-cluster installation	https://headlamp.dev/docs/latest/installation/in-cluster/
R52	Velero CNCF project status	https://presentations.cncf.io/projects/velero/
R53	Kubernetes Cloud Controller Manager	https://kubernetes.io/docs/concepts/architecture/cloud-controller/
R54	Proxmox Cloud Controller Manager	https://github.com/sergelogvinov/proxmox-cloud-controller-manager
R55	Kubernetes Metrics Server	https://github.com/kubernetes-sigs/metrics-server
R56	Kubernetes NodeLocal DNSCache	https://kubernetes.io/docs/tasks/administer-cluster/nodelocaldns/
R57	Kubernetes Descheduler	https://github.com/kubernetes-sigs/descheduler
R58	Kubernetes CSI Snapshot Controller	https://kubernetes-csi.github.io/docs/snapshot-controller.html